

CAMPUS EATS**Process Document**

Version 2.4 | Online REST API Edition | Full RBAC on Web and Mobile | Firebase Integration

PHP Primary Application | TypeScript Fallback Application | Android Application

Prepared for Rosebank International South Africa

Revision History

Version	Date	Summary of changes
2.3	Previous issue	Online REST API edition with full RBAC on the Android app and the PHP website.
2.4	1 October 2026	Added the two web application versions (PHP primary, TypeScript fallback built with Lovable) and the fallback procedure (sections 1.4, 8.3, 8.4). Added Firebase integration (sections 22, 9.3 and platform notes in sections 11, 13 to 20). Corrected the step numbering in section 14. Conclusion and references moved to sections 23 and 24.

Contents

1. INTRODUCTION	5
1.1 Purpose of the System	5
1.2 Problem Statement	5
1.3 Proposed Solution	5
1.4 Application Versions	6
2. ORGANIZATIONAL CONTEXT	8
2.1 Organization Overview	8
2.2 Business Objectives	8
3. STAKEHOLDER IDENTIFICATION	9
3.1 Primary Stakeholders	9
3.2 Secondary Stakeholders	9
4. SYSTEM SCOPE AND BOUNDARIES	10
4.1 In Scope	10
5. BUSINESS PROCESS OVERVIEW	11
5.1 Order Placement Process	11
5.2 Vendor Order Management Process	11
6. FUNCTIONAL REQUIREMENTS	12
6.1 Student and Standard User Functional Requirements	12
6.2 Vendor Functional Requirements	12
6.3 Administrator Functional Requirements	12
7. NON-FUNCTIONAL REQUIREMENTS	14
8. SYSTEM ARCHITECTURE	15
8.1 High-Level Architecture	15
8.2 Core System Modules	15
8.3 Client Implementations and Data Layers	16
8.4 Deployment and Fallback Strategy	16
9. DATABASE DESIGN OVERVIEW	17
9.1 Core Entities	17
9.2 Key Relationships	17
9.3 Additional Data Stores	18
10. USER INTERFACE DESIGN CONSIDERATIONS	19
10.1 Student and Standard User Screens	19
10.2 Vendor Screens	19
10.3 Administrator Screens	19

11. MOBILE USER ID SYSTEM.....	20
11.1 16-Character Global Identifier	20
11.2 Order ID System	20
12. USER ROLES (4 TYPES, 8 DEMO ACCOUNTS).....	21
12.1 Student Role	21
12.2 Standard Role	21
12.3 Vendor Role	21
12.4 Admin Role	21
12.5 Demo Accounts.....	22
13. ACCOUNT CREATION PROCESS	23
Step 1: User Registration	23
Step 2: Storage	23
Step 3: Output to User.....	23
Platform notes	23
14. ACCOUNT RECOVERY SYSTEM	24
15. LOGIN SYSTEM.....	25
16. SESSION MANAGEMENT	26
17. SECURITY IMPLEMENTATION.....	27
18. ORDER FLOW (END-TO-END)	28
19. ONLINE-FIRST BEHAVIOR	29
Data Access.....	29
Network Dependency.....	29
Real-Time Updates.....	29
Mobile Cache and Synchronization	29
20. SYSTEM INTEGRITY RULES	30
21. FEASIBILITY ANALYSIS	31
21.1 Technical Feasibility	31
21.2 Economic Feasibility.....	31
21.3 Operational Feasibility	31
22. FIREBASE INTEGRATION	32
22.1 Purpose.....	32
22.2 Firebase Project Configuration	32
22.3 Firebase Services Used	32
22.4 Realtime Database Structure	33
22.5 Synchronization on the Android App	33

22.6 Security Rules	34
22.7 Error Handling	34
22.8 Firebase Usage by Application Version.....	34
22.9 Open Alignment Items	36
23. CONCLUSION	37
24. REFERENCES.....	38

1. INTRODUCTION

1.1 Purpose of the System

Campus Eats is a food ordering and pickup platform built for Rosebank International South Africa. Customers order from registered campus vendors, vendors manage their menus and incoming orders, and administrators oversee the whole system.

There are three client applications, and all of them support all four roles: Student, Standard, Vendor, and Admin.

1. An Android app, usable by any role. A customer places orders from it, but a vendor can also log in and manage orders from their phone, and an admin can check reports on the go.
2. A PHP web application, which is the primary web application, usable by any role. A vendor or admin will likely prefer the larger screen for day-to-day management, but a student or standard user can just as easily log in and order from a browser.
3. A TypeScript web application, developed with Lovable, which is the fallback web application. It offers the same interface, functionality, and user behavior as the PHP web application and is used when the PHP version cannot be hosted or does not work as expected in the target environment (see section 1.4).

Which app someone uses is a matter of convenience, not a restriction tied to their role. Role-based access control is enforced by the backend, not by the client they happen to open. For data held in Firebase, access is additionally enforced by the Firebase Realtime Database security rules (see section 22).

1.2 Problem Statement

Ordering food at a physical stall on campus runs into the same problems most manual processes do:

1. Long queues during peak hours.
2. Miscommunication between students and vendors about what was actually ordered.
3. No digital record of orders.
4. Payments handled by hand.
5. No transaction history to check against.
6. Poor coordination between vendors and whoever is handling pickup.
7. Nothing tying ordering, payment, vendor management, and delivery together.

1.3 Proposed Solution

Campus Eats addresses this with a REST API that acts as the single source of truth for the system. The Android app and both web applications talk to the same backend services and expose the same functionality across all four roles, so a user's experience

depends on their role and permissions, not on which app they happen to open. Firebase Authentication and the Firebase Realtime Database provide managed sign-in and cloud data synchronization across devices (see section 22).

The solution covers:

1. A customer interface for browsing menus and placing orders.
2. A vendor portal for managing menu items and processing orders.
3. An administrative dashboard for system control and reporting.

All three of the above are available on the Android app and on both web applications (PHP and TypeScript), gated by the user's role rather than the platform.

1.4 Application Versions

The web application exists in two versions. Both are complete implementations of this document, and the Android app is the third client.

Version	Status	Technology	Purpose
PHP	Primary web application	PHP (7.3.12 or higher, 8.3 recommended) with MySQL 8.0.18 or higher	Default web deployment and reference implementation.
TypeScript	Fallback web application, developed with Lovable	TypeScript 5, React 18, Vite 5, Tailwind CSS	Alternative implementation that supports hosting requirements and provides a backup solution.
Android	Mobile application	Kotlin, Jetpack Compose, Room, Retrofit, Firebase	Mobile access for all four roles.

The TypeScript version was developed using Lovable, a vibe coding platform. It was created as an alternative implementation to support hosting requirements. If the PHP version cannot be hosted, or does not function as expected in the target environment, the TypeScript version can be used as a fallback. The deployment decision procedure is given in section 8.4.

The PHP and TypeScript versions have the same GUI, functionality, and user behavior. They are intended to provide an equivalent user experience, with no intentional differences in the interface, workflows, or application behavior. The choice between them is therefore an operational decision, not a functional one. Differences exist only below the user interface, in the data layer each version uses (see section 8.3).

Source code and live deployment:

Application	Location
Android (Kotlin)	https://github.com/HChristopherNaoyuki/campus-eats-app-kt.git
PHP (primary)	https://github.com/HChristopherNaoyuki/campus-eats-web.git
TypeScript (fallback)	https://github.com/HChristopherNaoyuki/campus-eats-platform.git
TypeScript live deployment (Lovable)	https://campus-eats-platform.lovable.app

2. ORGANIZATIONAL CONTEXT

2.1 Organization Overview

The system is meant for a college campus with several food vendors, a base of registered customers (students and standard users), and administrative staff. Any of these groups can work through the Android app or either web app, since all platforms carry the same role-based access model.

2.2 Business Objectives

The college wants to:

- Improve takeaway service for customers.
- Increase vendor operational efficiency.
- Improve revenue tracking and reporting.
- Reduce physical congestion at vendor stalls.
- Provide management with structured data insights.

3. STAKEHOLDER IDENTIFICATION

3.1 Primary Stakeholders

1. Students: browse menus, place orders, and pay, through the Android app or either website. They get student pricing.
2. Standard users: same flow on any platform, without the student discount.
3. Vendors: campus food stalls and cafeterias that prepare and fulfill orders, and manage listings and inventory, through the Android app or either website.
4. System administrator: manages users, vendors, reports, and configuration, through the Android app or either website, with full system and security oversight.

3.2 Secondary Stakeholders

1. Campus management.
2. IT support department.
3. Finance department.

4. SYSTEM SCOPE AND BOUNDARIES

4.1 In Scope

1. User registration and authentication for all four roles, on all clients.
2. Role-based access control (RBAC) enforced by the backend, consistently across all clients.
3. Menu browsing and search.
4. Cart management on any client.
5. Order placement and confirmation.
6. Order status tracking across all clients.
7. Administrative reporting, available on the Android app and on both websites.
8. Online-first operation. The system needs an internet connection to send and retrieve data from the API. The Android app also keeps a local cache (see section 19).
9. Firebase Authentication and Firebase Realtime Database integration for identity and cloud data synchronization (see section 22).
10. Two web application versions, PHP (primary) and TypeScript (fallback), with equivalent behavior, and a documented fallback procedure (see sections 1.4 and 8.4).

5. BUSINESS PROCESS OVERVIEW

5.1 Order Placement Process

1. Customer logs into the mobile app or either website.
2. Customer browses vendor menus, pulled live from the API.
3. Customer adds items to the cart.
4. Customer confirms the order and pays.
5. The order is sent to the API.
6. The vendor sees the new order come in, on whichever client they are using.
7. Vendor accepts or rejects it.
8. Vendor prepares the order.
9. Status is updated to "ready for pickup" through the API.

5.2 Vendor Order Management Process

1. Vendor logs into the mobile app or either website.
2. Vendor checks pending orders, pulled live from the API.
3. Vendor accepts or rejects the order.
4. Vendor updates preparation status.
5. Vendor marks the order ready for pickup.

6. FUNCTIONAL REQUIREMENTS

Each role's functionality below is available on the Android app and on both web applications (PHP and TypeScript). The backend enforces what a given role can do; the client just decides how it is presented.

6.1 Student and Standard User Functional Requirements

Students and standard users should be able to, on any client:

1. Register and log in.
2. Browse and search vendor menus.
3. Add items to a cart.
4. Modify what's in the cart.
5. Place orders.
6. Track order status.
7. View order history.
8. Recover their account using the registered email.

6.2 Vendor Functional Requirements

Vendors should be able to, on any client:

1. Register a vendor profile.
2. Add new menu items.
3. Update existing menu items.
4. Remove menu items.
5. Set item availability.
6. Accept or reject orders.
7. Update order status.
8. Track inventory per item.

6.3 Administrator Functional Requirements

Administrators should be able to, on any client:

1. Manage user accounts.
2. Manage vendor accounts.
3. View system reports.
4. Monitor transactions.
5. Deactivate or suspend accounts.
6. View security logs.

7. NON-FUNCTIONAL REQUIREMENTS

1. The system must support at least 1,000 concurrent users.
2. Response time must stay under 3 seconds per request.
3. Authentication must be secure for all four roles.
4. Data must move over HTTPS.
5. The system must run on Android devices and modern web browsers, with full functional parity across roles on all clients.
6. Data integrity and confidentiality must be maintained.
7. The system must operate online to send and retrieve data through the REST API.
8. The PHP and TypeScript web applications must provide equivalent GUI, functionality, and behavior, so that can be deployed without retraining users.
9. The TypeScript web application must be deployable as a fallback when the PHP web application cannot be hosted or does not function as expected.
10. Firebase access must be controlled by Firebase Authentication and Realtime Database security rules. Passwords and server-side credentials must not be stored in the database or in client code.

8. SYSTEM ARCHITECTURE

8.1 High-Level Architecture

The system is a distributed, RESTful setup with three layers, supported by Firebase cloud services (see section 22).

Backend layer (REST API): built on ASP.NET Core 8 with Entity Framework Core. This is the central server; all business logic, data access, and RBAC enforcement happen here, regardless of which client made the request. It runs on a central server, whether that is Azure, AWS, or something on-premise. Section 8.3 describes the data layer each current client implementation actually uses.

Client layers:

- Android app: built in Kotlin with Jetpack Compose (or an XML-based UI), using Retrofit or Ktor to talk to the API. The UI adapts based on the logged-in user's role, so a vendor sees menu and order management screens, an admin sees dashboards and logs, and a student or standard user sees the ordering flow.
- TypeScript web application (fallback): built in TypeScript using HTTP client to talk to the API, and developed with Lovable. Same principle: the interface shown depends on the user's role, not on an assumption baked into the platform.
- PHP web application (primary): built in PHP using HTTP client to talk to the API. Same principle: the interface shown depends on the user's role, not on an assumption baked into the platform.

Security layer: every authenticated request needs an API key (the user code).

Passwords are hashed, not stored in plain text. HTTPS is mandatory between clients and the API. Role checks happen server-side, so a client cannot grant itself permissions it was not given.

8.2 Core System Modules

1. Authentication and authorization module: handles registration, login, and API key generation for all roles.
2. User management module: handles user profiles and roles.
3. Vendor management module: handles vendor profiles.
4. Menu management module: handles menu items.
5. Order processing module: handles order creation, retrieval, and management.
6. Payment management module: simulated payment processing, available on any client.
7. Reporting module: analytics and reporting tools, available on any client for roles permitted to see them.
8. Cloud identity and synchronization module (Firebase): handles Firebase Authentication sign-in and Realtime Database synchronization for the clients that use them (see section 22.8).

8.3 Client Implementations and Data Layers

The three clients give users the same role-based experience. Underneath, each implementation stores and synchronizes data differently. This table reflects the source code of the three repositories listed in section 1.4.

Aspect	Android	PHP (primary)	TypeScript (fallback)
Primary data store	Room local database, synchronized to Firebase	MySQL, authoritative for accounts, authentication, orders, carts, and business data	Application state persisted in the browser, mirrored to Firebase for identity and profiles
Catalogue and order API	Fake Restaurant API through Retrofit	Fake Restaurant API over HTTPS	Fake Restaurant API through a hosted edge function that acts as a proxy
Firebase	Authentication, Realtime Database, Analytics	Configured, with client modules present. Not loaded by any active page in the reviewed code	Authentication, Realtime Database, Analytics (when configured)
Firebase configuration source	google-services.json file in the app module	Environment variables, with defaults in includes/firebase_config.php, served by api/firebase_config.php	Edge function firebase-config, with the key held in the backend secret store
Hosting	Installed on Android devices (API 24 or higher)	Web server with PHP and MySQL	Static single page application plus the edge functions, or the Lovable deployment

8.4 Deployment and Fallback Strategy

The PHP web application is the primary web deployment. The TypeScript web application is held ready as a fallback. The procedure is:

1. Deploy the PHP web application on a web server that meets its requirements (PHP with the PDO, MySQLi, JSON, Session, cURL, and OpenSSL extensions, and MySQL).
2. Verify the deployment against the role workflows in sections 5 and 6 and the first administrator account procedure in the PHP repository.
3. If PHP or MySQL cannot be provided, or the PHP application does not function as expected in the target environment, switch to the TypeScript web application. No change to the user interface, workflows, or training is needed, because both versions behave the same way.
4. The TypeScript web application can be used from the Lovable deployment, or built with `npm run build` and hosted as static files. When it is hosted outside Lovable, the edge functions it depends on (the API proxy and the Firebase configuration function) must be deployed with it.

Both web applications and the Android app use the same Firebase project where Firebase is enabled (see section 22), so switching between web versions does not require a change of Firebase project.

9. DATABASE DESIGN OVERVIEW

9.1 Core Entities

Entity	Key Fields	Notes
User	userEmail, passwordHash, role, userCode	role drives access control across all clients; userCode doubles as the API key
Restaurant	restaurantName, address, type, parkingLot	represents a vendor
Menu	itemName, itemPrice, itemDescription, imageUrl	linked to a Restaurant
Order	totalPrice, userDetails	master order; can contain one or more individual order records
Order_Items	orderId (foreign key)	items within a master order

9.2 Key Relationships

1. A user has one role (student, standard, vendor, or admin).
2. A user can have many orders.
3. A restaurant has many items on the menu.
4. A master order contains one or more individual orders.

9.3 Additional Data Stores

The entities above are held in different stores depending on the client. Section 22.4 describes the Firebase Realtime Database nodes in detail.

Store	Used by	Holds
MySQL	PHP	Users, authentication data, orders, carts, and business data (authoritative).
Room (local database)	Android	Local cache and offline records for users, menu items, cart items, orders, feedback, coupons, and debit cards.
Browser persisted state	TypeScript	Session and application state kept in the browser.
Firebase Realtime Database	Android, TypeScript	Users, feedback, and (Android only) orders and coupons. See section 22.4.
Fake Restaurant API	All clients	Restaurant and menu catalogue data and related API calls.

10. USER INTERFACE DESIGN CONSIDERATIONS

The Android app and both web applications present role-appropriate screens. What follows is organized by role, not by platform, since each role's screens exist on every client. The PHP and TypeScript web applications share the same GUI.

10.1 Student and Standard User Screens

- Home screen: shows all vendors and available items.
- My Orders screen: shows receipts and order history.
- Cart screen: shows everything the user wants to buy, with the fee, tax, and discount rules below.

Rule	Detail
Service fee	Under 500 rand: 10%. Between 500 and 1,000 rand: 6.5%. 1,000 rand and above: no fee.
Tax	20%
Rounding	Round up to the nearest 5 rand
Student discount	2.5% off, applied only to the Student role

- Feedback screen: lets the user submit a compliment or complaint.

10.2 Vendor Screens

- Home screen: vendor summary.
- Menu screen: CRUD operations for menu items.
- Orders screen: order management.
- Reports screen: sales analytics.

10.3 Administrator Screens

- Home screen: system statistics.
- User Management screen.
- Vendor Management screen.
- Order Management screen.
- Transactions screen.
- Reports screen.
- Security Log screen.

All interfaces follow consistent navigation and usability standards, regardless of which platform they are rendered on.

11. MOBILE USER ID SYSTEM

11.1 16-Character Global Identifier

Every user, regardless of role, gets a unique, immutable 16-character alphanumeric ID at registration.

- Format: XXXX-XXXX-XXXX-XXXX (stored as one continuous string).
- Example: A9F3-KL21-ZX90-QWER.
- Generated with a cryptographically secure random generator.
- Required for account recovery.
- Used for cross-device identity restoration, including switching between the Android app and the websites.
- Used as the user's key under the users node in Firebase. The Android app writes the continuous 16-character string. The TypeScript web application converts it to the hyphenated 19-character form (for example ABCD1234EFGH5678 becomes ABCD-1234-EFGH-5678) and converts it back when reading. See section 22.9.

11.2 Order ID System

Every order gets a unique ID at checkout.

- Format: ORD-YYYYMMDD-XXXXXXXXXX.
- Example: ORD-20260627-8F3A91BC.
- Globally unique and immutable once created.
- Linked to the user's ID.

12. USER ROLES (4 TYPES, 8 DEMO ACCOUNTS)

Each role below works on the Android app and on both web applications. There is no platform restriction tied to roles.

12.1 Student Role

- Access campus vendors.
- Browse menus.
- Place orders.
- View order history.
- Receive student-specific pricing.

12.2 Standard Role

- Full consumer experience.
- Cart management.
- Checkout and order tracking.
- Order modification before confirmation.

12.3 Vendor Role

- Manage product listings.
- Accept or reject orders.
- Update order status: Pending → Preparing → Ready → Completed.
- Inventory tracking per item.
- View sales analytics.

12.4 Admin Role

- Full system control.
- Manage users and vendors.
- View system logs.
- Audit orders.
- Security monitoring dashboard.

12.5 Demo Accounts

The following demo accounts are provided to facilitate testing and demonstration of the various system roles. The local database is pre-populated with these authoritative records on initial launch.

Role	Name	Email	Password
Administrator	Amara Nkosi	amara.nkosi@campuseats.test	Admin#Amara
Administrator	Pieter van Wyk	pieter.vanwyk@campuseats.test	Admin#Pieter
Vendor	Thandiwe Mokoena	thandiwe.mokoena@campuseats.test	Vendor#Thandi
Vendor	Sipho Dlamini	sipho.dlamini@campuseats.test	Vendor#Sipho
Vendor	Annelie Botha	annelie.botha@campuseats.test	Vendor#Annelie
Standard	Lerato Khumalo	lerato.khumalo@campuseats.test	Standard#Lerato
Standard	Johan Pretorius	johan.pretorius@campuseats.test	Standard#Johan
Standard	Zanele Ndlovu	zanele.ndlovu@campuseats.test	Standard#Zanele
Standard	Marius Steyn	marius.steyn@campuseats.test	Standard#Marius
Student	Naledi Mahlangu	naledi.mahlangu@campuseats.test	Student#Naledi

13. ACCOUNT CREATION PROCESS

Step 1: User Registration

User enters email, password, and role. Registration works the same way whether it happens on the Android app or on either website. The system generates a 16-character user ID, hashes the password with BCrypt or Argon2, and sets the account status to active by default.

Step 2: Storage

The record goes into the central database through the API, indexed by user ID and email for fast lookup.

Step 3: Output to User

The user sees their 16-character ID and is told to store it somewhere safe, since it is the key to recovering the account on any platform.

Platform notes

- **PHP:** the account is stored in MySQL, which is authoritative for accounts and authentication.
- **TypeScript:** the account is created in the application and mirrored to Firebase Authentication and to the users node. Firebase holds the credential, and the database record carries only the marker [FIREBASE_SSO] in place of a password hash.
- **Android:** registration is local first. The account is saved to the local Room database flagged as not yet synchronized, and the background synchronizer writes it to the users node when the device is online (see section 22.5). Registration through Google sign-in is written to Firebase immediately.

14. ACCOUNT RECOVERY SYSTEM

Recovery runs on the 16-character user ID and works the same way regardless of which client the user is on.

1. User selects "Recover Account".
2. Users enter their ID.
3. The system checks it against the API.
4. If it matches, identity is verified.
5. User sets a new password or confirms biometric authentication.
6. Password reset only happens after verification.

Security rule: the user ID alone is not sufficient to log in. It must be paired with a password, biometric authentication, or a session token.

Platform note: the Android app can also send a Firebase Authentication password reset email to the registered email address, which supports the requirement that users can recover their account using the registered email (section 6.1).

15. LOGIN SYSTEM

Login needs the 16-character user ID and a password, on any client. The password hash comparison happens on the API server. On success, a session token is issued with an expiry attached.

Platform notes:

- **Android:** the user signs in with email and password. The app checks the local record first. If it does not match, the app signs in through Firebase Authentication (internet required) and loads the user's profile from the users node into the local cache. Google sign-in is available through Firebase Authentication. Suspended accounts are refused.
- **TypeScript:** the account may be identified by email address, username, or 16-character user ID. The app signs the user in to Firebase Authentication, creating the Firebase account on first use. Google sign-in is available through Firebase Authentication.
- **PHP:** authentication is handled by the PHP application against MySQL. Google sign-in uses the Google OAuth 2.0 authorization code flow, and the application then looks up or creates the matching MySQL account.

16. SESSION MANAGEMENT

Sessions live on the client and are validated against the API. They expire after 15 to 60 minutes of idle time. Logging out clears the local session record, and expired tokens cannot be reused. A user's session on the Android app is independent of any session they hold on the website.

Platform note: on the Android app and the TypeScript web application, logging out also signs the user out of Firebase Authentication.

17. SECURITY IMPLEMENTATION

- Password hashing: BCrypt or Argon2.
- All traffic over HTTPS.
- Role-Based Access Control (RBAC): gates both the UI and the underlying logic on all clients.
- Anti-tampering: hash verification on critical entities.
- Secure token generation for sessions.
- Input validation on all data sent to the API.
- All actions logged server-side, tagged with the role and platform that performed them.
- Firebase security rules deny access by default and grant it per node (see section 22.6).
- Passwords are never written to the Realtime Database. The passwordHash field holds only the marker [FIREBASE_SSO].
- Administrator rights in Firebase are determined by an administrator custom claim on the signed-in user's token, not by anything the client sends.
- Firebase web configuration values are public client identifiers, not secrets. They are not reproduced in this document. Server-side administrator credentials must never be placed in client code or in a source repository.

Implementation note: the PHP application hashes passwords with the PHP `password_hash` function. The Android app hashes the password stored in its local record with SHA-256 and relies on Firebase Authentication for cloud credentials. SHA-256 does not meet the BCrypt or Argon2 requirement above and is recorded as an open item in section 22.9.

18. ORDER FLOW (END-TO-END)

1. User adds items to the cart, stored locally on whichever client they are using.
2. Checkout starts.
3. An Order is created through a POST request to the API.
4. The API generates a master order ID.
5. Inventory is updated on the server.
6. The local cart is cleared.
7. The order is added to the user's history on the API.

Platform note: on the Android app the order is also stored in the local Room database flagged as pending synchronization, and the background synchronizer writes it to the orders node in Firebase (see section 22.5).

19. ONLINE-FIRST BEHAVIOR

The system depends on an active connection to the REST API for essentially everything that reads or changes shared data, on the Android app and on both websites.

Data Access

When any client opens, it fetches current data from the API, whether that is restaurant listings, menus, or order status, and the UI is populated from that response. Anything that changes data, like placing an order or updating a profile, is sent to the API right away.

Network Dependency

A stable connection is required to use any client. The system checks for connectivity and tells the user when it is missing, for example, "No internet connection. Please check your network settings." If a request fails due to a network issue, the system shows an error and lets the user retry.

Real-Time Updates

The UI updates as responses come back from the API. Where real-time status matters, this can be handled with polling or WebSockets, on any client.

Mobile Cache and Synchronization

The Android app adds a local layer on top of the online-first model:

- A local Room database caches data on the device.
- Firebase Realtime Database disk persistence is enabled with a 10 MB cache, so recently loaded data can be shown while the connection is weak.
- New registrations are saved locally first and synchronized to Firebase later.
- A background synchronizer skips its cycle while the device is offline and retries with increasing delays after a failure (see section 22.5).
- Operations that need the authoritative server state, such as profile updates, bank account linking, shop status changes, and administrator actions, still require an internet connection.

20. SYSTEM INTEGRITY RULES

- Every entity is tied to a user ID.
- No orphan orders are allowed.
- No anonymous transactions are permitted.
- Every action is logged on the API server, regardless of which client or role initiated it.
- On the Android app, a local record is marked as synchronized only after Firebase confirms the write.
- Firebase records are keyed by the user ID, order ID, or coupon code, so repeating a write does not create duplicates.

21. FEASIBILITY ANALYSIS

21.1 Technical Feasibility

Nothing here requires unusual technology: ASP.NET Core 8 with Entity Framework Core on the backend, a native Android client in Kotlin with Retrofit, and a PHP web client, with a TypeScript web client as a fallback. Firebase is a managed service, so no additional server is needed for identity or cloud synchronization. Since all clients consume the same API and enforce the same RBAC rules server-side, extending full role support to every platform does not add architectural complexity, mainly UI work on each client to present the right screens per role. The TypeScript version has lighter hosting needs than the PHP version, which is the reason it serves as the fallback. No special hardware is required.

21.2 Economic Feasibility

The cost is mostly development time. Against that, the system should pay off in operational efficiency and clearer financial tracking. Maintaining two web versions adds some effort to keep them equivalent. This is accepted in exchange for lower deployment risk.

21.3 Operational Feasibility

Both customers and vendors are already used to mobile and web-based ordering, so training needs should be minimal. Giving vendors and admins the option of either client, instead of locking them into the website, should if anything make onboarding easier. Because the PHP and TypeScript versions behave the same way, switching to the fallback needs no additional training.

22. FIREBASE INTEGRATION

22.1 Purpose

Firebase provides managed user authentication and a cloud Realtime Database. In Campus Eats it is used for sign-in (including Google sign-in), for synchronizing user, order, feedback, and coupon records between devices, and as a cloud backup of records created on the Android app. The Android app and the TypeScript web application use the same Firebase project. The PHP web application is configured for the same project but is MySQL-authoritative (see section 22.8).

22.2 Firebase Project Configuration

The values below are public identifiers taken from the project configuration. API keys are intentionally not reproduced here.

Item	Value
Project ID	campus-eats-db
Realtime Database URL	https://campus-eats-db-default-rtdb.europe-west1.firebaseio.com
Database region	europe-west1
Authentication domain	campus-eats-db.firebaseio.com
Storage bucket (configured)	campus-eats-db.firebaseio.com
Android application ID	com.example.campus_eats_app_kt
Android SDK	Firebase BoM with Analytics, Authentication, and Realtime Database
Web SDK	firebase 12.18.0 (pinned in the TypeScript version, and the default version in the PHP version)
Sign-in methods	Email and password, Google sign-in

Configuration is supplied to each client as follows: the Android app reads google-services.json, the TypeScript application requests its configuration from the firebase-config edge function, and the PHP application serves its configuration from api/firebase_config.php using environment variables. Web API keys for Firebase are client identifiers rather than secrets, but they should be restricted in the Google Cloud console, and access control must rely on Firebase Authentication and the security rules.

22.3 Firebase Services Used

- **Firebase Authentication:** email and password sign-in, Google sign-in, password reset email (Android), and the administrator custom claim.

- **Realtime Database:** cloud storage of the nodes in section 22.4, secured by the rules in section 22.6.
- **Analytics:** included in the Android app, and started by the TypeScript application when a measurement ID is configured.

22.4 Realtime Database Structure

Node	Written by	Content
users/{userId}	Android, TypeScript	userId, fullName, username, email, role (STUDENT, STANDARD, VENDOR, or ADMIN), passwordHash (always [FIREBASE_SSO]), status, walletBalance, shopName, shopStatus, bankAccountInfo, registrationDate, usercode. The TypeScript application writes a smaller profile (userId, fullName, email, role, passwordHash, status, firebaseUid).
orders/{orderId}	Android	orderId, customerId, vendorId, itemsJson, totalAmount, status, paymentMethod, pickupTime, specialRequests, timestamp.
feedback/{feedbackId}	Android, TypeScript	userId (the Firebase user), type (complaint or compliment), subject, message, userName, userEmail, status (pending or resolved), createdAt, updatedAt.
coupons/{code}	Android	code, discountPercent, isActive, expiryDate, assignedUserId.

Fields such as walletBalance and bankAccountInfo are sensitive. Read access to a user record is limited by the rules to the record owner and administrators.

22.5 Synchronization on the Android App

The Android app uses a single background synchronizer between the local Room database and Firebase. Its behavior is:

1. Records are saved locally first and flagged as pending synchronization.
2. A loop runs about every 10 seconds and checks for an internet connection before it writes.
3. Pending users, pending orders, feedback for the signed-in user, and coupons are written to their nodes using the record key, so repeated writes are idempotent.
4. A local flag is cleared only after Firebase confirms the write.
5. After a failure, the loop waits 1, 2, 4, 8 seconds and so on, up to a maximum of 60 seconds, and returns to the normal interval after a successful cycle.

6. Only one loop runs at a time.

Firebase disk persistence is enabled with a 10 MB cache. The performance notes in the Android repository also describe keeping the signed-in user's profile node synchronized for fast reads.

22.6 Security Rules

Access is controlled by the Realtime Database security rules. As defined in the rules files of the web repositories, the intent is:

- Deny all reads and writes by default.
- users: a signed-in user may read and write only the record whose email matches their authenticated email. Administrators may read and write all records.
- users: the fields `userId`, `fullName`, `email`, and `role` are required, `role` must be one of the four allowed values, and `passwordHash` must equal `[FIREBASE_SSO]`.
- users: `walletBalance` cannot be negative, and a non-administrator cannot increase it. The `status` field can be changed only by an administrator.
- feedback: a signed-in user may create a record with their own Firebase user ID, type `complaint` or `compliment`, and `status` `pending`. A user may read only their own feedback. Administrators may read all feedback and mark it `resolved`.
- Indexes: users on `email`, `role`, and `status`. Feedback on `userId`, `type`, `status`, and `createdAt`.

Administrator rights are decided from the administrator custom claim on the user's token.

22.7 Error Handling

Firebase failures are never treated as successes. The Android app translates Firebase errors into plain messages for the user, covering invalid credentials, an existing account, a disabled account, an expired session, too many attempts, network failure, and permission denied. The TypeScript application records Firebase sign-in and profile synchronization results in its security log and shows the last Firebase error to the user.

22.8 Firebase Usage by Application Version

Capability	Android	PHP (primary)	TypeScript (fallback)
Firebase Authentication	Yes (sign-in fallback, Google sign-in, password reset email)	Yes	Yes (signs in or creates the account at login and registration)

Capability	Android	PHP (primary)	TypeScript (fallback)
Google sign-in	Through Firebase	Through Firebase	Through Firebase
user's node	Yes	Yes	Yes
orders node	Yes	Yes	Yes
feedback node	Yes	Yes	Yes
coupons node	Yes	Yes	Yes
Local store	Room database	MySQL	Browser persisted state
Password handling	Local SHA-256 hash, Firebase Authentication for cloud credentials	password_hash in MySQL	Firebase Authentication holds the credential

This table describes how each implementation stores and synchronizes data. It does not describe the user interface. The PHP and TypeScript versions present the same GUI, functionality, and user behavior to the user (see section 1.4).

22.9 Open Alignment Items

The following items were found when the three repositories were compared. They should be resolved before the Firebase project is used as the shared production service.

1. User key format: the Android app writes 16-character continuous keys, the TypeScript application writes 19-character hyphenated keys, and the rules files differ. The TypeScript rules accept only 19 characters, and the PHP repository rules accept 16 or 19. One canonical key form and one deployed rules file are required.
2. Rules coverage: the rules files in the web repositories define users and feedback only, with all other paths denied. The orders and coupons nodes written by the Android app need matching rules, otherwise those writes are refused.
3. Administrator rights: the TypeScript rules use the administrator custom claim, while the PHP repository rules use an admin_claims node. One approach should be chosen and deployed consistently.
4. Password hashing: the Android local record uses SHA-256, which does not meet the BCrypt or Argon2 requirement in section 17.
5. PHP Firebase client: the PHP repository contains Firebase client scripts and a configuration endpoint that no active page loads. A decision is needed on whether the PHP application should use Firebase or remain MySQL only.

23. CONCLUSION

Campus Eats is a scalable food ordering and pickup system built around a REST API, with an Android app and two web applications that all support the four roles: Student, Standard, Vendor, and Admin. The PHP web application is the primary web version. The TypeScript web application, developed with Lovable, is an equivalent fallback that can be used if the PHP version cannot be hosted or does not work as expected. Firebase Authentication and the Firebase Realtime Database provide managed sign-in and cloud synchronization. The Fake Restaurant API (AravindAttepu, n.d.) gives it a realistic backend to work against for the purposes of demonstrating the design.

What the system covers:

1. Clearly defined stakeholders.
2. Structured business processes.
3. A full set of functional requirements.
4. A clean separation between backend, mobile client, and web clients, with consistent RBAC enforced across all of them.
5. Online-first operation with REST API integration.
6. A complete RBAC model across four roles, available on all platforms.
7. Real interaction with a live RESTful API.
8. Two equivalent web versions, PHP (primary) and TypeScript (fallback), with a documented fallback procedure.
9. Firebase integration for identity and cloud synchronization, with documented security rules and open alignment items.

24. REFERENCES

AravindAttepu, n.d. Fake Restaurant API. [Online]
Available at: <https://fakerestaurantapi.runasp.net/>
[Accessed 24 July 2026].

HChristopherNaoyuki, n.d. campus-eats-app-kt (Android application). [Online]
Available at: <https://github.com/HChristopherNaoyuki/campus-eats-app-kt>
[Accessed 1 October 2026].

HChristopherNaoyuki, n.d. campus-eats-web (PHP web application). [Online]
Available at: <https://github.com/HChristopherNaoyuki/campus-eats-web>
[Accessed 1 October 2026].

HChristopherNaoyuki, n.d. campus-eats-platform (TypeScript web application). [Online]
Available at: <https://github.com/HChristopherNaoyuki/campus-eats-platform>
[Accessed 1 October 2026].

Campus Eats Platform, n.d. TypeScript web application deployed with Lovable. [Online]
Available at: <https://campus-eats-platform.lovable.app>
[Accessed 1 October 2026].